

Auditoria Técnica — Cur10usX

Data: Maio 2026 | Auditor: Tech Lead | Âmbito: Full-stack + DevOps

Resumo Executivo

O projeto Cur10usX encontra-se num estado **funcional e estável**, com uma stack moderna e bem integrada. A arquitetura monolítica modular é adequada ao contexto atual. No entanto, existem áreas críticas que precisam de atenção antes de considerar o projeto "pronto para produção e equipa".

Score Geral: 7.5/10 — Sólido, com oportunidades de melhoria focadas.

Dimensão	Score	Estado
Arquitetura	8/10	✅ Boa
Base de Dados	9/10	✅ Excelente
Backend (API)	8/10	✅ Boa
Frontend (UI/UX)	7/10	⚠️ Boa, mas inconsistente
Segurança	6/10	⚠️ Atenção necessária
DevOps	7/10	⚠️ Boa, com issues
Testes	2/10	❌ Crítico
Documentação	8/10	✅ Boa
Performance	6/10	⚠️ Sem otimizações

1. Arquitetura — Pontos Fortes

✅ Monolito Modular Bem Estruturado

- Separação clara entre UI components, API routes, libs, providers e tipos
- Route groups do Next.js App Router bem utilizados para organizar secções

- Single source of truth no Prisma schema para tipos de BD
- Core libraries em `src/lib/` bem modularizadas (auth, audit, rate-limit, etc.)

✓ Multi-tenant Robusto

- Modelo de dados com schoolId em todas as entidades
- Catálogo global → escola → local (3 níveis de abstração)
- Feature flags por escola (`School.features` JSON field)

✓ Sistema de Permissões Completo

- 5 roles com hierarquia clara
- 15 permissões granulares para school_admins
- Session version para invalidar tokens

✓ Motor de Avaliação Configurável

- GlobalGradingConfig com overrides por escola
- Pesos por trimestre, fórmulas configuráveis, modos de arredondamento
- Suporte a recurso/exame de recurso

2. Arquitetura — Pontos Fracos

✗ Acoplamento Next.js + WebSocket

- WebSocket server embutido no mesmo processo (entrypoint.sh lança `node ws-server.js &`)
- Se o WS falha, o container continua (sem supervisão)
- WS server não partilha sessão com Next.js — autenticação fraca (só userId)
- Sem Redis pub/sub — não escala horizontalmente

✗ Rate Limiting em Memória

- `src/lib/rate-limit.ts` usa Map em memória
- Perde dados ao reiniciar servidor
- Não funciona com múltiplas instâncias

✗ Sem Cache Layer

- Todas as páginas fazem queries à BD

- Landing page (página inicial) faz 6 queries a cada request
- Dashboard de admin faz múltiplos counts que podem ser lentos com muitos dados

✗ Sem Testes

- Zero testes unitários, de integração, ou E2E
- Risco alto para regressões ao integrar equipa

3. Base de Dados

✓ Bom Schema

- 30+ modelos bem normalizados
- Índices adequados nos campos de lookup
- Enums para estados (evita magic strings)
- @@unique constraints garantem integridade

⚠ Questões

- `Result` sem unique constraint (pode haver duplicados)
- `AuditLog` sem cascade delete (pode crescer indefinidamente)
- `directUrl` no datasource duplicado com `url`
- Migrations incluem `migration_lock.toml` que deve estar em git

4. Frontend — UI/UX

✓ Bom Sistema de Componentes

- 32+ componentes reutilizáveis em `src/components/ui/`
- 20 formulários CRUD padronizados
- Sistema de paginação, pesquisa, filtros

✗ Inconsistências Identificadas

Área	Problema	Impacto
Landing Page	Boa estrutura mas faltam animações, transições, micro-interações	Perceção de qualidade

Área	Problema	Impacto
Responsividade	Prováveis problemas em mobile (tabelas largas, layouts fixos)	Acessibilidade mobile
Loading States	Alguns componentes podem não ter skeleton/spinner	UX pobre em redes lentas
Error States	Handle de erros pode ser inconsistente entre páginas	Debugging difícil
Empty States	Listas sem dados mostram tabelas vazias vs mensagens informativas	UX confusa
Acessibilidade	Sem aria-labels, focus management, keyboard navigation	Exclusão de utilizadores
Dark Mode	Pode ter contraste insuficiente em alguns componentes	Legibilidade
Form Validation	Inconsistência entre validação client-side e server-side	UX frustrante
Animações	Transições entre páginas podem ser abruptas	Percepção de performance

✗ Organização Frontend

- `src/components/ui/` contém componentes de domínio (ex: `StudentDashboard.tsx`) misturados com UI genéricos
- Alguns componentes têm demasiadas responsabilidades
- Falta separação entre containers (smart) e presentationais (dumb)

5. Segurança — Questões Críticas

🔴 CRÍTICO: Secrets no Repositório

- `.env` no root e em `cur10us/.env` contêm:
 - Database URL ativa com password em texto plano
 - Google OAuth client secret
 - Resend API key
 - Vercel Blob token
 - `AUTH_SECRET` (chave de assinatura JWT)

Ação imediata: Rotacionar todos os secrets e remover do git history.

🟡 Médio: WebSocket Auth Fraca

- `ws-server.js` aceita qualquer `userId` sem verificar sessão

- Qualquer cliente autenticado pode ouvir notificações de outro userId

🟡 Médio: Sem Helmet/CSP

- Headers de segurança HTTP não configurados
- Sem Content-Security-Policy (risco XSS teórico)

🟢 Bom: Proteções Existentes

- CSRF tokens em operações de mutação
 - Rate limiting em auth endpoints
 - Open redirect validation no middleware
 - bcrypt para password hashing
 - Session version para invalidação
 - 2FA implementado
-

6. DevOps

✅ Docker

- Multi-stage build (deps → builder → runner)
- Non-root user no container
- Healthcheck configurado
- Docker secrets para credentials

❌ Kubernetes

- `apiVersion: cur10usx/v1` — inválido. Deve ser `apps/v1`
- **Label mismatch**: deployment `app: cur10usx`, service selector `app: cur10usx-app`
- Sem liveness probe (apenas readiness)
- Sem resource limits (apenas requests)

❌ CI/CD

- Pipeline apenas com lint + type-check
- Sem step de build (pode compilar mas falhar em runtime)
- Sem testes
- Sem deploy automático

⚠ Docker Compose

- Sem serviço Redis (apesar de `REDIS_URL` estar configurada)
- Sem serviço de banco de dados local (depende de Neon externo)

7. Performance

Gargalos Atuais

Problema	Ocorre em	Impacto
Múltiplos counts no dashboard	Admin dashboard	Lento com milhares de registos
Sem paginação server-side em algumas listas	Páginas de listagem	Lento com muitos dados
Sem lazy loading	Todas as páginas	Bundle inicial grande
Sem ISR/SSG	Landing page, termos	TTFB alto
WebSocket connection por página	Todas as páginas autenticadas	Conexões duplicadas

Oportunidades

- Implementar `React.lazy()` para componentes pesados (charts, calendário)
- Usar ISR para landing page e páginas públicas
- Implementar paginação server-side consistente
- Adicionar Redis cache para queries frequentes

8. Débito Técnico

Item	Esforço	Impacto	Prioridade
Secrets no repositório	1h	🔴 Crítico	Imediata
Zero testes	40h+	🔴 Alto	Curto prazo
K8s configs inválidas	30min	🟡 Médio	Curto prazo
WebSocket auth fraca	4h	🟡 Médio	Curto prazo
Organização de componentes	8h	🟢 Baixo	Médio prazo
Landing page polishing	16h	🟢 Baixo	Médio prazo
Responsividade mobile	20h	🟢 Baixo	Médio prazo

Item	Esforço	Impacto	Prioridade
Redis infra	8h	● Médio	Longo prazo
Performance dashboard	12h	● Médio	Longo prazo

9. Oportunidades de Modularização

Componente Atual	Problema	Sugestão
<code>src/components/ui/</code> (mix UI + domínio)	Falta separação clara	Criar <code>src/components/domain/</code> para componentes de negócio
<code>src/app/api/[resource]/route.ts</code>	Código repetitivo CRUD	Criar helpers genéricos de CRUD
<code>src/app/(dashboard)/list/[entity]/</code>	Muita duplicação entre listas	Criar template genérico de lista
Landing page sections	Dados mockados vs dados reais	Standardizar data fetching
WebSocket	Acoplado ao Next.js	Extrair para serviço standalone

10. Problemas de UX/UI — Checklist Detalhado

Landing Page

- ☐ Faltam animações de scroll (reveal on scroll)
- ☐ Hero section sem CTA secundário claro
- ☐ Secção de preços pode não ter plano gratuito visível
- ☐ Footer sem links rápidos para auth

Dashboard

- ☐ Loading state pode ser apenas spinner genérico
- ☐ Empty state mostra tabela vazia em vez de mensagem + CTA
- ☐ Error state pode não ter ação de retry

- ☐ Feedback de ações (create/edit/delete) pode ser inconsistente

Formulários

- ☐ Validação client-side pode não corresponder à server-side
- ☐ Success/error toast não implementado globalmente
- ☐ Confirmação de delete pode ser inconsistente

Tabelas/Listagens

- ☐ Responsividade horizontal em ecrãs pequenos
- ☐ Ordenação pode não ter indicador visual claro
- ☐ Filtros podem não ter estado "limpar tudo"
- ☐ Paginação em mobile pode ser difícil de usar

Geral

- ☐ Acessibilidade: teclado, screen readers, focus trap em modais
- ☐ Contraste de cores no dark mode
- ☐ Feedback tátil/hover em todos os elementos interativos
- ☐ Transições de página suaves
- ☐ Skeleton loading em vez de spinner

11. Análise de Segurança Detalhada

Checklist OWASP Top 10

Risco	Estado	Notas
A1: Broken Access Control	✓	requireRole/requirePermission implementados
A2: Cryptographic Failures	⚠	Secrets commitados — rotação necessária
A3: Injection	✓	Prisma previne SQL injection
A4: Insecure Design	✓	Rate limiting, CSRF, validação
A5: Security Misconfiguration	⚠	Sem CSP/HSTS headers
A6: Vulnerable Components	✓	Dependencies atualizadas
A7: Auth Failures	⚠	WS auth fraca
A8: Data Integrity Failures	✓	Session version

Risco	Estado	Notas
A9: Logging Failures	✓	AuditLog implementado
A10: SSRF	✓	Sem funcionalidade de fetch URLs

12. Métricas do Projeto

Métrica	Valor
Linhas de schema Prisma	952
Modelos de dados	30+
Enumerações	17
API endpoints	40 grupos / 154 route.ts
Componentes UI	32+
Formulários CRUD	20
Ficheiros de landing	11
Libs core	22
Hooks custom	4
Providers React	3
Schemas Zod	9
Línguas suportadas	4
Migrações DB	5
Commits Git	22
Tags	3 (v0.0.1, v1.0.0, v1.1.0)
Testes	0
Cobertura CI	Lint + type-check apenas

13. Prioridades de Ação

Imediatas (esta semana)

1. Rotacionar todos os secrets e remover do git history

2. Corrigir K8s manifests (apiVersion + labels)
3. Adicionar testes unitários aos módulos core (auth, validation, evaluation-engine)
4. Corrigir WebSocket auth para verificar sessão

Curto Prazo (1-2 semanas)

5. Adicionar testes de integração às API routes principais
6. Implementar landing page com animações e responsividade
7. Adicionar loading/error/empty states consistentes
8. Configurar CI/CD com build e deploy automático
9. Adicionar Redis ao docker-compose

Médio Prazo (3-4 semanas)

10. Refatorar organização de componentes (ui vs domain)
11. Implementar sistema de design consistente (tokens, variáveis)
12. Melhorar responsividade mobile
13. Adicionar acessibilidade (aria labels, keyboard nav)
14. Implementar cache layer com Redis

Longo Prazo (2+ meses)

15. Extrair WebSocket para serviço standalone
 16. Implementar background jobs (Bull + Redis)
 17. Separar domínio de avaliação em microsserviço
 18. Dashboard performance optimization
 19. E2E testing com Playwright/Cypress
-